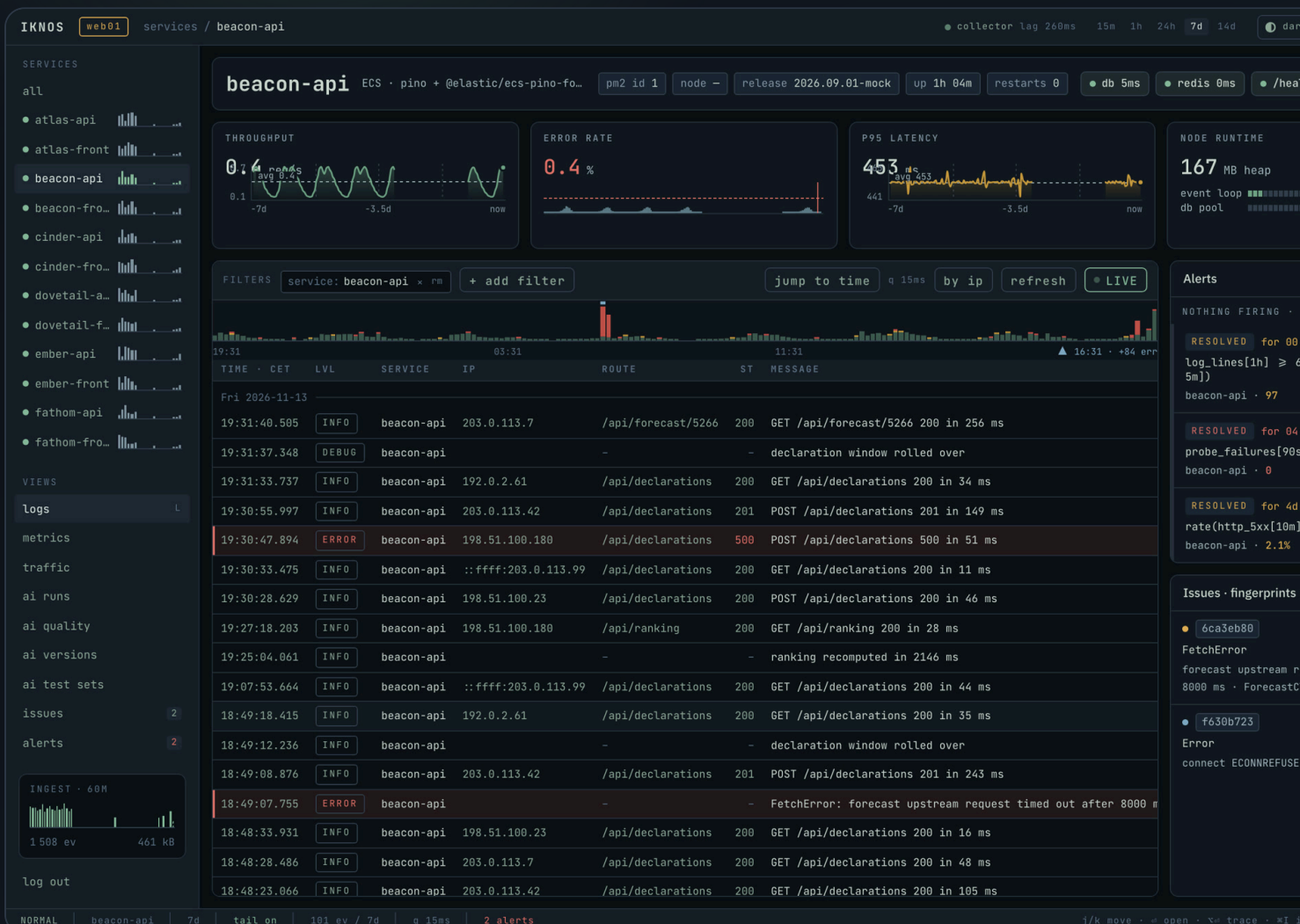
CONSOLE DE MONITORING

mode d'emploi

Logs, métriques, trafic, erreurs groupées, alertes et exécutions de pipelines IA : tout ce que la console sait faire, écran par écran, bouton par bouton.



VERSION DU 13 NOVEMBRE 2026 · CAPTURES SUR LA FLOTTE DE DÉMONSTRATION web01

00 Iknos en une page

Iknos est une console de monitoring auto-hébergée pour une petite flotte d'applications tournant sur une seule machine. Elle lit ce que les applications écrivent déjà — elle ne leur demande rien.

CE QU'ELLE MONTRE

Les logs en direct, la santé et les métriques de chaque service, le trafic des sites, les erreurs regroupées en *issues*, les alertes, et les exécutions des pipelines IA.

COMMENT ON S'EN SERT

On choisit un **service** dans le rail, une **vue** juste en dessous, une **plage de temps** en haut. Tout l'écran répond à ces trois choix.

CE QU'ELLE NE FAIT PAS

Aucune notification n'est envoyée nulle part : pas de mail, pas de webhook. On vient regarder. Un seul compte, une seule session active.

SOMMAIRE

01	**Comment ça marche**	d'où viennent les données	09	**Trafic**	visiteurs, pages, sources
02	**Se connecter**	compte unique, récupération	10	**Exécutions IA**	runs, cascade, inspecteur
03	**Anatomie de l'écran**	les zones, le modèle mental	11	**Qualité, versions, jeux de test**	suivre un pipeline IA
04	**La barre du haut**	machine, collecteur, plage, thème	12	**Issues**	erreurs regroupées
05	**Le rail**	services, vues, ingestion	13	**Alertes**	règles, états, actions
06	**Logs**	filtres, histogramme, direct	14	**Palette et clavier**	⌘K et tous les raccourcis
07	**Vue service**	identité, santé, quatre signaux	15	**Lire les états vides**	ce que chaque message veut dire
08	**Métriques**	routes, percentiles	16	**Mener une enquête**	de l'alerte à la ligne fautive

CONVENTIONS DE CE DOCUMENT

L'interface est en anglais : ses libellés sont cités tels quels, dans une étiquette comme `add filter`. Les touches sont notées `⌘` `K` ; sur un clavier non-Mac, `Ctrl` remplace `⌘` partout. Les pastilles jaunes (1) renvoient aux repères posés sur les captures. Les captures montrent la flotte de démonstration (machine `web01`, services inventés).

01 Comment ça marche

Une application surveillée ne connaît pas Iknos. Elle écrit ses logs sur sa sortie standard et expose ses métriques ; Iknos lit ce qui existe déjà sur la machine.

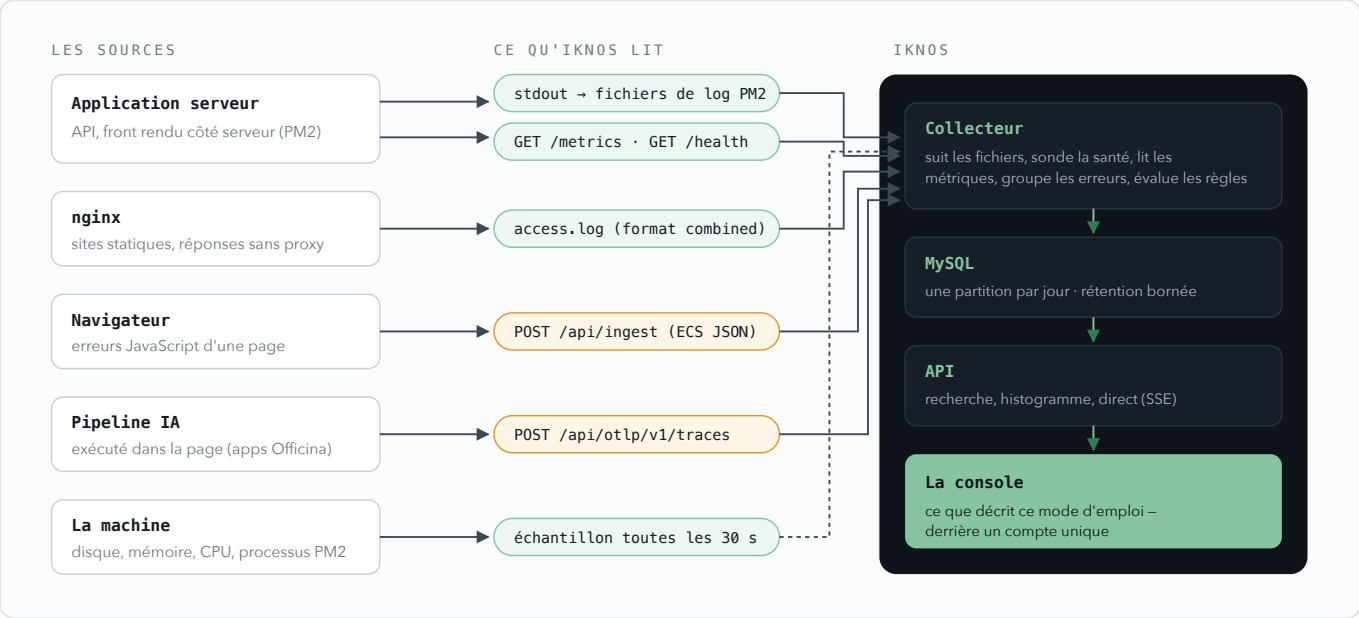


SCHÉMA 1 En vert, ce qu'Iknos va chercher lui-même sur la machine. En orange, les deux seuls cas où un programme envoie quelque chose : une page web (qui n'a pas de sortie standard) et un pipeline IA (dont l'exécution est un arbre d'étapes minutées, pas une ligne de log).

Ce que cela implique à l'écran

- **Un log est une ligne ECS** : horodatage, niveau, service, route, statut HTTP, durée, message, identifiant de trace.
- **Les lignes d'une même requête** partagent un *trace id* ; la console sait les réunir.
- **Les erreurs identiques** sont regroupées par empreinte (*fingerprint*) en une *issue*.
- **Les règles d'alerte** sont évaluées par le collecteur, à cadence fixe.
- **Iknos se surveille lui-même** par le même chemin : il apparaît comme un service parmi les autres.

Ce qui est conservé, et combien de temps

Les logs et les spans sont rangés par jour et supprimés au-delà de la rétention (14 jours par défaut). Une plage plus ancienne n'affiche donc pas zéro : elle n'existe plus. Le contenu des appels de modèle (prompts, réponses) a sa propre durée de vie, plus courte.

> **À RETENIR**
> Une vue dont la donnée n'existe pas est **vide et le dit**, jamais remplie de valeurs factices. Le chapitre 15 traduit chacun de ces messages.

02 Se connecter

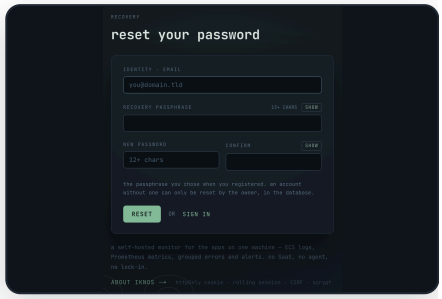
Une instance Iknos a un seul compte. L'inscription est ouverte tant qu'aucun compte n'existe, puis se referme définitivement.



CONNEXION `sign in` : e-mail et mot de passe.



SESSION EXPIRÉE Le bandeau dit pourquoi vous êtes revenu ici.



RÉCUPÉRATION `reset your password` par phrase secrète.

ÉCRAN	À QUOI IL SERT	À SAVOIR
`sign in`	Ouvrir la console. Après connexion, vous arrivez sur les logs.	Un mauvais mot de passe et une adresse inconnue donnent le même message, `invalid credentials`. Après trop d'essais : attendre une minute.
`create an account`	Créer *le* compte de l'instance (lien `REGISTER`).	Mot de passe de 12 caractères minimum, avec jauge `STRENGTH`, et phrase de récupération de 13 caractères minimum. Dès que le compte existe, l'écran affiche `this instance already has its account` et le formulaire est grisé.
`reset your password`	Choisir un nouveau mot de passe (lien `RECOVER ACCOUNT →`).	Il n'y a pas d'e-mail de récupération : seule la phrase secrète choisie à l'inscription fonctionne. Notez-la. Après trop d'essais : attendre quinze minutes.
`legal notice`	Mentions légales (lien `ABOUT IKNOS →`).	Seule page lisible sans compte, avec les trois précédentes.

LA SESSION

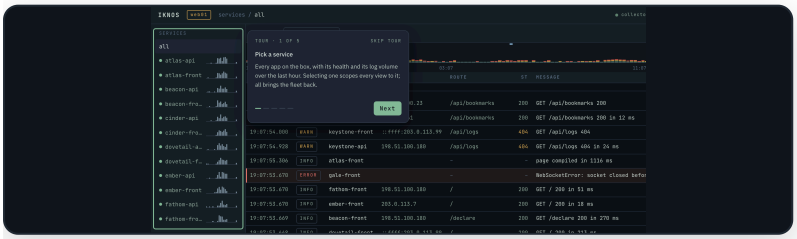
Elle glisse : chaque action repousse son échéance, qui ne mesure donc que l'inactivité (douze heures). Quand elle a expiré, la console vous renvoie à la connexion avec `session expired · sign in again` plutôt que d'afficher une erreur.

UNE SEULE SESSION À LA FOIS

Se connecter ailleurs (autre navigateur, autre machine) ferme la session précédente. Si votre onglet repasse soudain à l'écran de connexion, c'est qu'une autre connexion a eu lieu avec le même compte.

Se déconnecter : le lien `log out`, tout en bas du rail, ou `⌘` `⇧` `L`.

La visite guidée du premier lancement



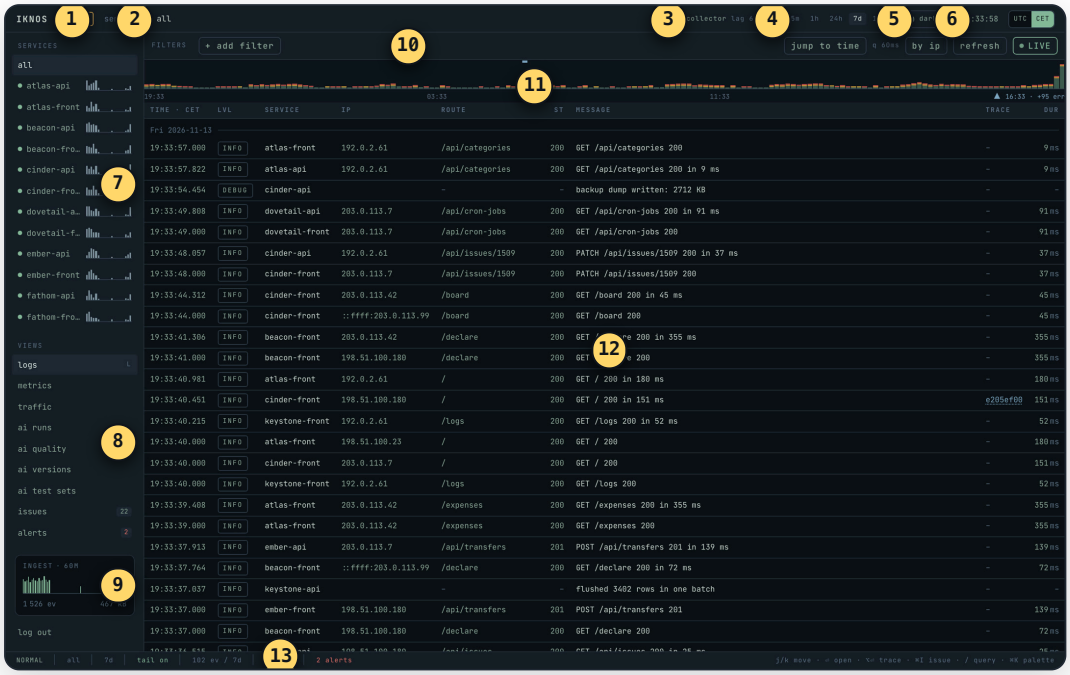
VISITE Étape 1 sur 5 : la zone concernée est encadrée, la carte l'explique.

À la première ouverture dans un navigateur, une carte `Tour · 1 of 5` présente cinq zones du cadre : les **services**, les **vues**, la **plage de temps**, le badge de la **machine** et la carte d'**ingestion**.

`Next` avance, `Done` termine, `Skip tour` la ferme pour de bon : elle ne revient pas dans ce navigateur.

03 Anatomie de l'écran

Un cadre fixe — barre du haut, rail à gauche, barre d'état en bas — et une surface de travail qui change avec la vue. La page elle-même ne défile jamais : seules les listes défilent.



CAPTURE 1 La vue `logs` sur toute la flotte (`all`), plage de sept jours.

1. **Badge machine** — le nom de l'hôte ; sa couleur est son niveau d'alerte. Un clic ouvre le panneau machine.
2. **Fil d'Ariane** — le périmètre courant : `services / all` ou le service choisi.
3. **Collecteur** — pastille d'état et retard (`lag`) de la collecte.
4. **Plage de temps** — de 15 minutes à 14 jours, valable pour toutes les vues.
5. **Thème** — clair ou sombre pour la surface de travail.
6. **Horloge et fuseau** — l'heure courante, et le fuseau dans lequel tout est affiché.
7. **Services** — toutes les applications de la machine, avec leur santé et leur activité.
8. **Vues** — ce que l'on regarde : logs, métriques, trafic, IA, issues, alertes, avec leurs compteurs.
9. **Carte d'ingestion** — ce que le collecteur a lu en une heure. Un clic ouvre stockage et rétention.
10. **Barre de filtres** — les filtres actifs et les commandes de la liste.
11. **Histogramme** — le volume de lignes sur la plage, coloré par niveau.
12. **Surface de travail** — ici le flux de logs ; ailleurs un tableau, des cartes, une cascade.
13. **Barre d'état** — mode, périmètre, plage, direct, volume, durée de requête, alertes, rappel des touches.

Le modèle mental : trois choix, un écran

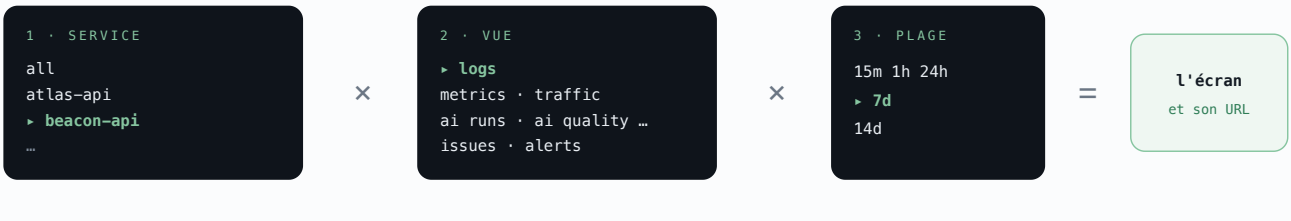


SCHÉMA 2 Le service, la vue, la plage et les filtres voyagent dans l'adresse de la page : copier l'URL, c'est partager exactement ce que l'on voit. Changer de vue garde le service et la plage.

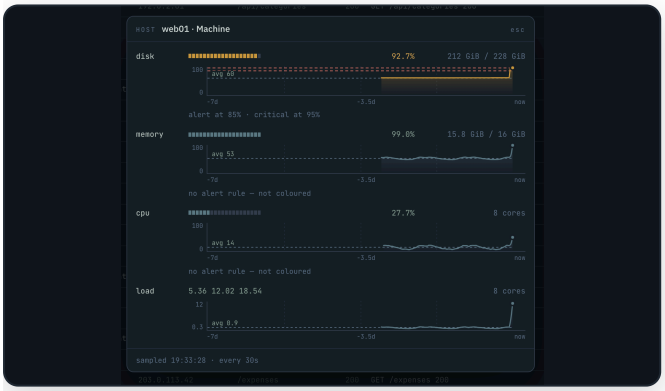
04 La barre du haut

Elle ne change jamais d'une vue à l'autre. De gauche à droite :



ÉLÉMENT	CE QU'IL DIT	CE QU'IL FAIT AU CLIC
`web01`	**La machine.** Bordure neutre : dans les limites. Ambre : seuil d'avertissement dépassé. Rouge : seuil critique. Au survol, le niveau en toutes lettres et le chiffre en cause.	Ouvre le panneau `Machine` (voir plus bas).
`services / all`	**Le périmètre** : toute la flotte, ou le nom du service choisi.	–
`collector lag 615ms`	**Le collecteur.** collecte (`collecting`) · en retard (`behind`) · arrêté (`stopped`) · démarrage (`starting`). `lag` est le retard entre l'écriture d'une ligne et son arrivée en base.	Au survol : l'état et l'âge de la dernière vérification.
`15m 1h 24h 7d 14d`	**La plage de temps** : `15m` `1h` `24h` `7d` `14d`. Sept jours par défaut.	Change la fenêtre lue par toutes les vues, graphiques et compteurs.
`dark`	**Le thème** de la surface de travail. Les barres du cadre restent sombres dans les deux cas.	Bascule `dark` ↔ `light` ; le choix est mémorisé dans le navigateur.
`UTC CET`	**Le fuseau.** La moitié allumée est l'horloge sur laquelle tous les horodatages sont lus : `UTC` ou votre fuseau local (ici `CET`). L'en-tête de la colonne `time` le rappelle.	Bascule entre les deux. L'heure affichée à gauche suit.

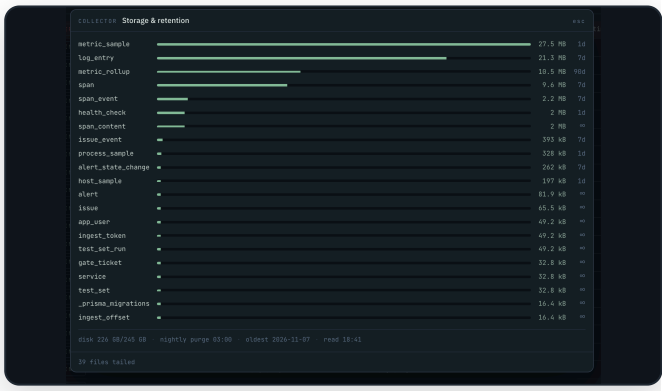
Le panneau machine CLIC SUR LE BADGE



CAPTURE 2 Quatre jauges et leur courbe sur la plage.

- `disk` – seule jauge surveillée par une règle d'alerte : les deux lignes pointillées sont ses seuils (`alert at 85% · critical at 95%`), et la jauge se colore quand elle les franchit.
- `memory` , `cpu` – sans règle, donc volontairement grises (`no alert rule — not coloured`).
- `load` – charge moyenne sur 1, 5 et 15 minutes, et nombre de cœurs.
- En bas : l'heure du dernier échantillon (un toutes les 30 secondes). `esc` ferme.

Stockage et rétention CLIC SUR LA CARTE D'INGESTION



CAPTURE 3 Chaque table, son poids sur disque et sa durée de conservation.

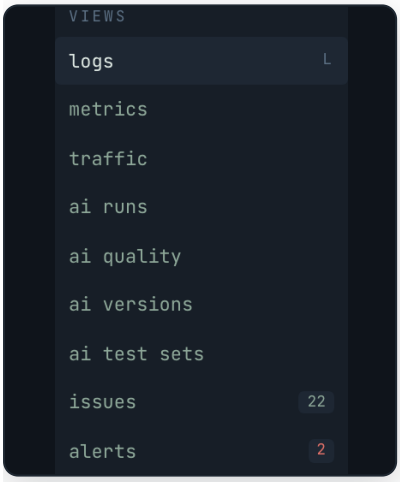
- Une ligne par table, de la plus lourde à la plus légère ; à droite sa rétention (`1d` , `7d` … ou `∞` quand rien n'est purgé).
- En pied : l'occupation du disque, l'heure de la purge nocturne, la date de la plus ancienne partition, l'heure de lecture, et le nombre de fichiers suivis (`files tailed`).

05 Le rail

La colonne de gauche porte les deux premiers choix – le service et la vue – et la carte d'ingestion.



SERVICES



VUES



INGESTION

Services

Une ligne par application de la machine. `all`, en tête, rend la flotte entière. Cliquer un service **restreint toutes les vues à ce service**, et la vue logs devient alors la *vue service* (chapitre 7).

SIGNE	SENS
pastille verte	La dernière sonde de santé a répondu (`healthy`).
pastille rouge	La sonde échoue (`failing`).
pastille ambre	Plus de sonde récente (`stale`, au-delà de 90 secondes).
petit graphique	Le volume de logs du service sur la dernière heure, quelle que soit la plage choisie.
`paused`	La collecte est désactivée pour ce service.
survol	Une bulle donne l'état, l'âge de la dernière sonde et sa latence. La liste défile dans son propre cadre.

Vues

VUE	CE QU'ELLE RÉPOND	CHAP.
`logs`	Que s'est-il écrit, ligne par ligne ?	6–7
`metrics`	Quelle route coûte sa latence au service ?	8
`traffic`	Qui visite les sites, d'où, et quelles pages ?	9
`ai runs`	Qu'ont fait les pipelines IA, exécution par exécution ?	10
`ai quality`	La qualité d'un pipeline tient-elle dans le temps ?	11
`ai versions`	Une version de prompt fait-elle mieux qu'une autre ?	11
`ai test sets`	Quelles exécutions ai-je mises de côté comme cas de test ?	11
`issues`	Quelles erreurs reviennent, regroupées ?	12
`alerts`	Quelle règle est déclenchée, ou l'a été ?	13

Le nombre à droite de `issues` compte les issues non résolues du périmètre ; celui de `alerts`, en rouge, les alertes en cours – le même chiffre que dans la barre d'état. À zéro, le compteur disparaît.

La carte d'ingestion

`INGEST · 60M` trace ce que le collecteur a lu sur soixante minutes : nombre d'événements (`ev`) et volume. Elle signale aussi les lignes perdues (`dropped`) ou illisibles (`unreadable`). Un clic ouvre le panneau `Storage & retention` du chapitre 4.

06 Logs

La vue d'arrivée. Trois étages : la barre de filtres, l'histogramme du volume, le flux des lignes — la plus récente en haut.

La barre de filtres



COMMANDE	RÔLE
`+ add filter` `/`	Ouvre le tiroir de filtre : on choisit un champ, une valeur, puis `add filter`. Champs : `service`, `level` (niveau minimal), `route`, `status`, `text` (mot cherché dans le message), `ip`, `trace`.
étiquette de filtre	Chaque filtre actif devient une étiquette `text: timeout`. Cliquer l'étiquette le **désactive en gardant sa valeur** (un second clic le réactive) ; la croix `×` le retire.
`jump to time`	Ouvre un champ date-heure ; `go` épingle la liste sur cet instant.
`q 52ms`	La durée de la dernière requête. Une recherche devenue lente se voit ici sans la chercher.
`by ip`	Remplace l'histogramme par le décompte des requêtes par adresse cliente.
`refresh`	Relance la requête sur la plage.
`● LIVE`	Suit les nouvelles lignes à mesure qu'elles arrivent (voir « Le direct »).



CAPTURE 4 Le tiroir `ADD FILTER` : le champ à gauche, sa valeur à droite.

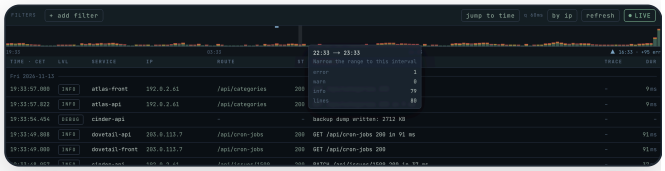


CAPTURE 5 Le filtre `text: timeout` posé : le flux *et* l'histogramme se resserrent.

L'histogramme

Chaque barre est un intervalle de la plage ; sa hauteur est le nombre de lignes, ses couleurs la répartition par niveau : ● erreur, ● avertissement, ● information.

- **Survol** : une bulle détaille l'intervalle (`error`, `warn`, `info`, `lines`).
- **Le repère ▲** à droite (`▲ 16:31 · +87 err`) désigne l'intervalle dont le nombre d'erreurs dépasse le plus le reste de la fenêtre. Le petit trait bleu au-dessus d'une barre le situe.
- **Clic sur une barre** : la liste se resserre sur cet intervalle. Un badge `PINNED WINDOW` apparaît ; `back to the range` revient à la plage.
- **Au clavier** : `←` `→` d'une barre à l'autre, `Home` / `End` aux extrémités.



CAPTURE 6 La bulle d'un intervalle.



CAPTURE 7 Fenêtre épinglée après un clic sur une barre.

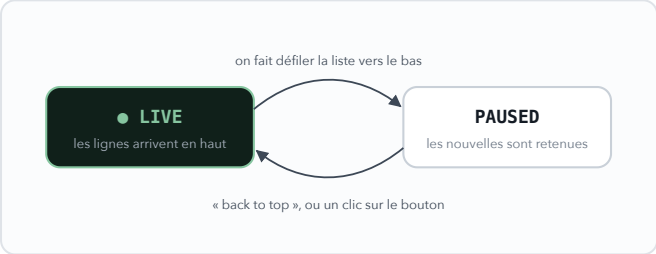
Le flux de lignes

COLONNE	CONTENU

`time · cet`	Heure de la ligne, dans le fuseau choisi (rappelé dans l'en-tête). Un séparateur de date marque chaque changement de jour.
`lvl`	Niveau : `DEBUG` `INFO` `WARN` `ERROR` `FATAL`. Les lignes d'erreur ont un fond rouge et un liseré à gauche.
`service`	L'application qui a écrit la ligne.
`ip`	L'adresse de l'appelant. Vide sur la plupart des lignes : seules celles qui en rapportent une (nginx surtout) la remplissent.
`route` · `st`	Le chemin de la requête et son statut HTTP.
`message`	Le texte de la ligne.
`trace`	Début de l'identifiant de trace, souligné quand il existe. **Cliquer ici ouvre la trace**, pas le détail.
`dur`	Durée mesurée ; au survol, la valeur exacte en millisecondes.

Le flux défile dans son propre cadre et charge les lignes plus anciennes en arrivant en bas. La fin est annoncée : `That is every line in the range.`

Le direct



En pause, une barre annonce `12 new lines` : rien n'est perdu, rien ne bouge sous vos yeux. Si le direct ne suit plus, une ligne le dit : `N lines dropped — the tail could not keep up`.

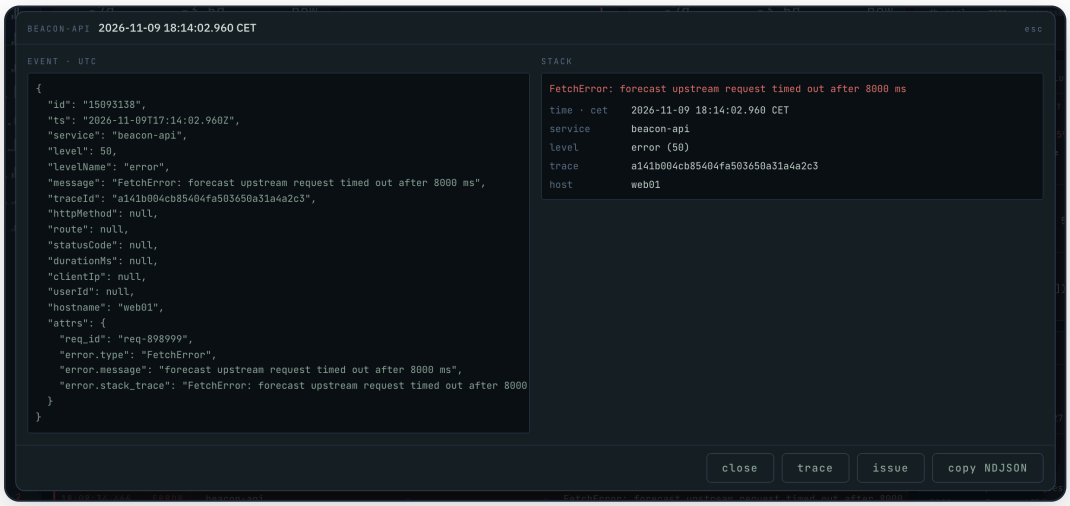
Grouper par adresse BY IP



CAPTURE 8 Requêtes par adresse cliente, sous les mêmes filtres.

Colonnes `ip`, `hits`, `routes`, `last`. Sur chaque adresse : `copy`, `whois` (à qui elle appartient et où — seule action qui interroge un service extérieur ; la réponse est mise en cache), et un filtre « ne montrer que cette adresse ». La liste est plafonnée aux adresses les plus actives et le signale.

Ouvrir une ligne : le détail CLIC SUR LA LIGNE · ⏎



CAPTURE 9 À gauche l'événement brut tel qu'il a été reçu (toujours en UTC) ; à droite les champs lisibles et, pour une erreur, la pile d'appels.

Le pied de la fenêtre porte quatre boutons :

- `close` – fermer (`esc`).
- `trace` – ouvrir la trace de cette ligne.
- `issue` – ouvrir l'issue dans laquelle cette erreur a été groupée.
- `copy NDJSON` – copier la ligne brute dans le presse-papiers.

Quand la ligne porte une adresse cliente, le champ `client` propose aussi la copie et le `whois`.

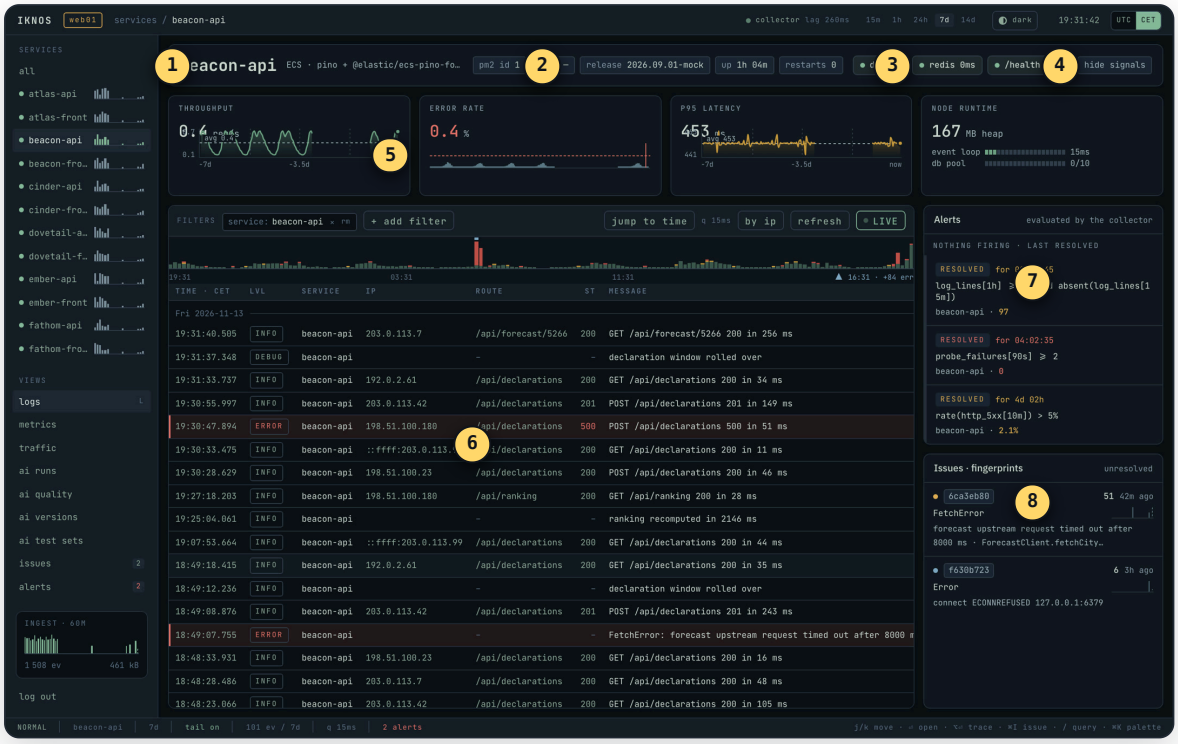
La trace CLIC SUR LA COLONNE TRACE · ⌥⏎



Toutes les lignes partageant le même *trace id*, dans l'ordre, tous services confondus : une requête suivie de bout en bout. Chaque ligne indique son décalage depuis le début (`offset`) et ce qu'elle a mesuré (`took`, ou `not measured`). `copy id` copie l'identifiant, `open in logs` filtre le flux sur cette trace.

07 Vue service

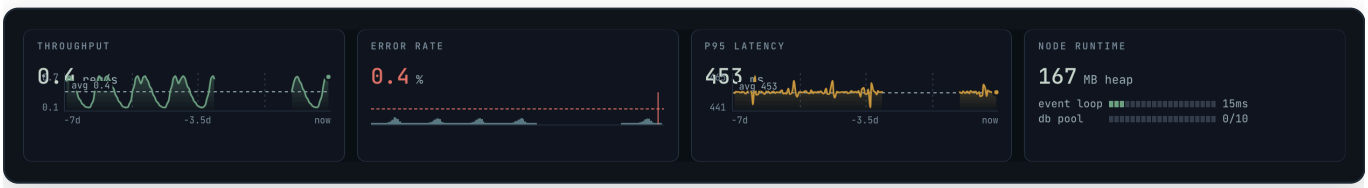
Dès qu'un service est choisi dans le rail, la vue logs s'enrichit : son identité et sa santé en tête, quatre signaux, puis ses logs, ses alertes et ses issues côte à côte.



CAPTURE 10 Le service `beacon-api` sur sept jours.

1. **Nom et émetteur** – le service, et le contrat de logs attendu de toute application (ECS sur la sortie standard).
2. **Identité du processus** – `pm2 id`, `node`, `release`, `up` (durée de fonctionnement), `restarts`.
3. **Pastilles de santé** – une par dépendance sondée (`db`, `redis`…) et le point `/health`, avec latence ou statut.
4. **`hide signals`** – replie les quatre tuiles pour donner la hauteur aux logs (`⌘` `L`).
5. **Quatre signaux** – débit, taux d'erreur, latence p95, runtime Node.
6. **Ses logs** – le flux du chapitre 6, déjà filtré sur `service: beacon-api`.
7. **Alertes du service** – ce qui est déclenché ; sinon les dernières résolues (`nothing firing · last resolved`).
8. **Issues du service** – les erreurs non résolues, par empreinte ; un clic ouvre l'issue.

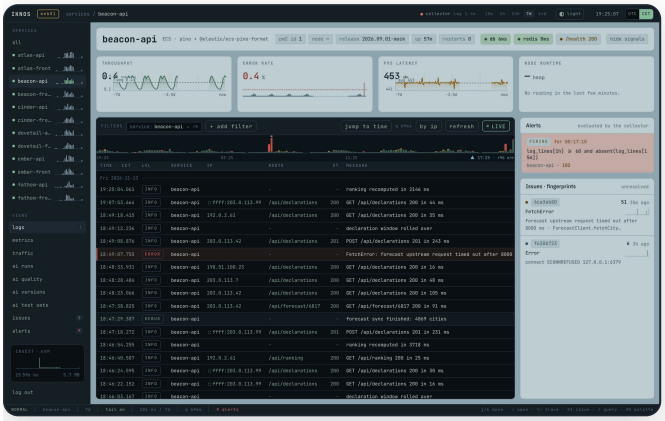
Les quatre signaux



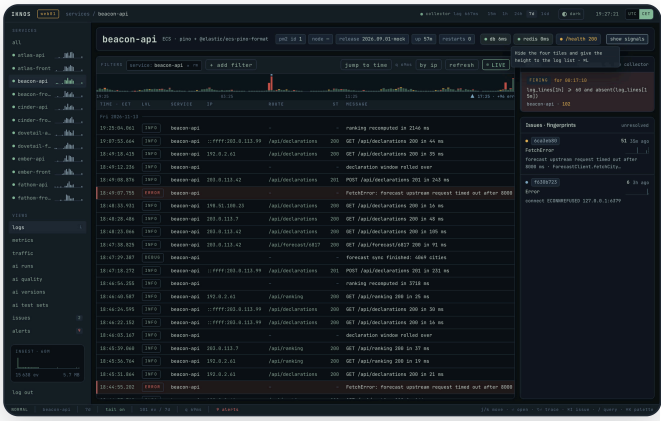
TUILE	CE QU'ELLE MESURE	AU CLIC
`THROUGHPUT`	Requêtes par seconde, toutes routes et tous statuts confondus. La ligne pointillée est la moyenne sur la plage.	Ouvre le tableau des routes (`metrics`).
`ERROR RATE`	Part des réponses 5xx parmi toutes les réponses – et seulement les 5xx.	Ouvre les logs d'erreur du service sur la plage.
`p95 LATENCY`	Le temps sous lequel passent 95 % des requêtes, avec sa valeur de référence sur toute la plage.	Ouvre le tableau des routes.

`NODE RUNTIME`	Mémoire `heap`, retard de la boucle d'événements (`event loop`, 99e centile) et occupation du pool de connexions (`db pool`, en service / taille ; la bulle dit combien de requêtes attendent).

Survoler une courbe affiche la valeur à cet instant. Survoler une pastille de santé donne son statut, sa latence, l'heure de la dernière sonde et sa cadence (toutes les 30 secondes) ; cliquer une pastille en échec ouvre les logs d'erreur autour de la sonde ratée. La puce `restarts` est le seul symptôme de la rangée : elle compte les redémarrages depuis que PM2 connaît le processus.



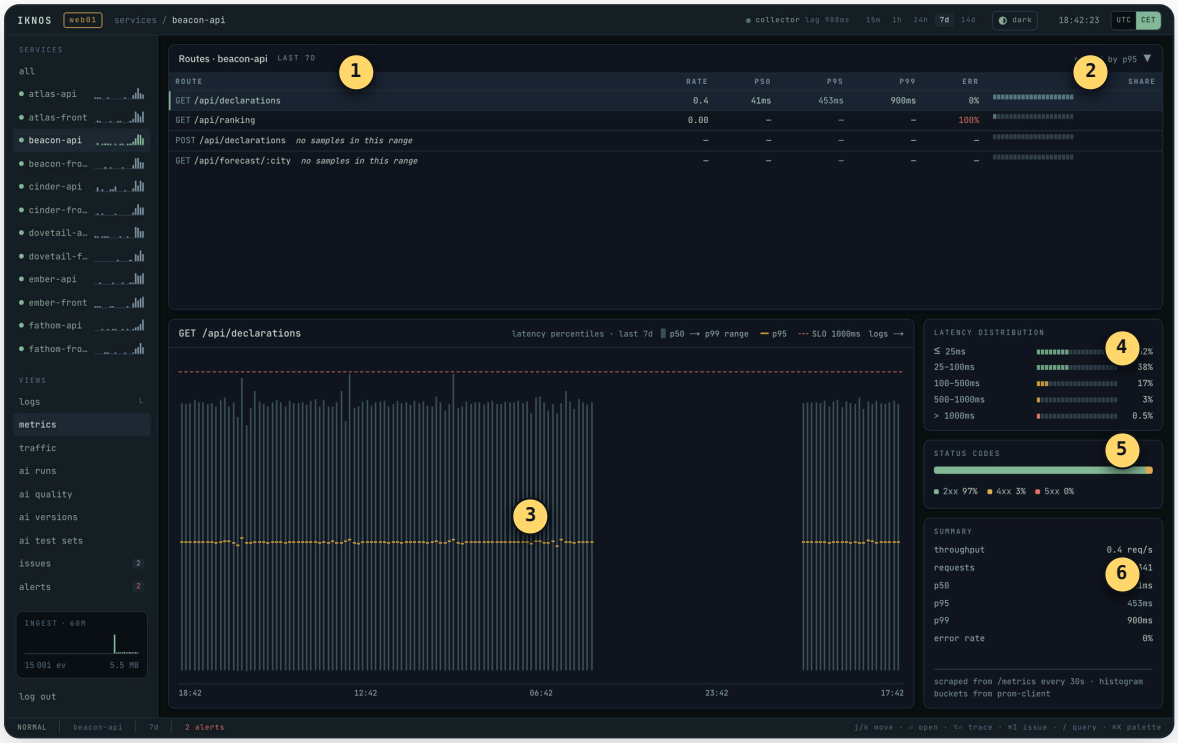
CAPTURE 11 Le thème clair : seule la surface de travail change, le cadre reste sombre. Ici une alerte `FIRING` occupe le panneau de droite.



CAPTURE 12 Signaux repliés (`show signals` les rend) : toute la hauteur va aux logs.

08 Métriques

Les routes d'un service, classées par latence. La question à laquelle répond cette vue : laquelle coûte son p95 au service ?



CAPTURE 13 Les routes de `beacon-api`, et le détail de la route sélectionnée.

1. **Tableau `Routes`** – trié par p95 décroissant. Colonnes : `route`, `rate` (req/s), `p50`, `p95`, `p99`, `err` (taux d'erreur).
2. **`share`** – la part de cette route dans les requêtes du service, en barre.
3. **Percentiles dans le temps** – p50, p95 et p99 sur la plage, avec la ligne `SLO` (le seuil de la règle d'alerte). Le lien `logs →` ouvre les lignes de cette route.
4. **`latency distribution`** – la répartition des requêtes par tranche de durée.
5. **`status codes`** – la répartition 2xx / 4xx / 5xx.
6. **`summary`** – débit, nombre de requêtes, percentiles et taux d'erreur de la route.

MODE D'EMPLOI

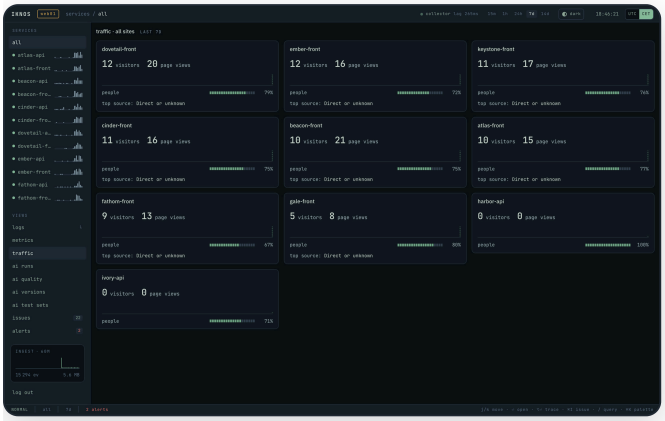
Choisir un service dans le rail (la vue le demande sinon : `Pick a service in the rail`), puis cliquer une route dans le tableau pour afficher son détail en dessous.

LIRE LES PERCENTILES AVEC JUSTESSE

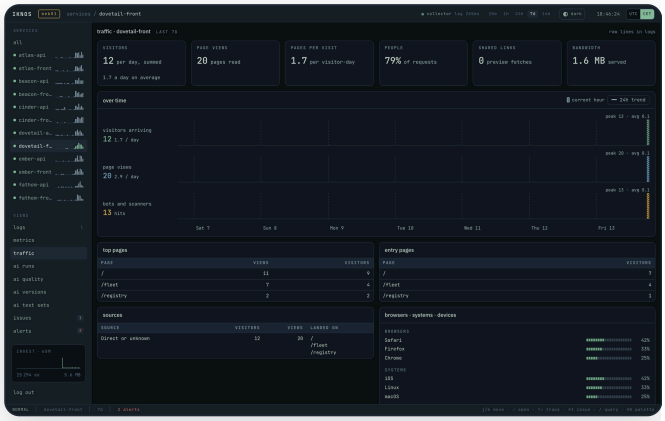
Ils sont **interpolés** à partir des tranches d'un histogramme, pas mesurés requête par requête. La ligne de provenance, en bas à droite, rappelle la cadence de lecture de `/metrics`. Une route connue mais jamais appelée sur la plage affiche `no samples in this range`.

09 Trafic

L'audience des sites servis par nginx, calculée à partir de ses journaux d'accès. Aucun traceur dans les pages, aucun cookie.



CAPTURE 14 Avec `all` : une carte par site — visiteurs, pages vues, part d'humains, source principale. Un clic ouvre le site.



CAPTURE 15 Avec un site choisi : tuiles de synthèse, courbes, puis tableaux. Cette vue défile.

Les six tuiles

TUILE	DÉFINITION EXACTE
`visitors`	Une adresse et un navigateur, sur un jour calendaire, additionnés sur la plage. La même personne sur deux jours compte deux fois : ce ne sont pas des « visiteurs uniques ».
`page views`	Chargements complets de page par des personnes. Images, scripts, robots et navigation interne à l'application ne sont pas comptés.
`pages per visit`	Pages vues divisées par visiteurs-jours.
`people`	Part des requêtes venant de navigateurs, face aux robots, outils et scanners.
`shared links`	Nombre d'aperçus récupérés par une messagerie ou un réseau social : quelqu'un y a posté un lien.
`bandwidth`	Volume servi.

Les blocs sous les tuiles

- `over time` — trois séries : visiteurs arrivant, pages vues, robots et scanners. Le bouton à droite bascule entre l'heure courante et la tendance sur 24 h ; les flèches du clavier parcourent les intervalles.
- `top pages`, `entry pages` — les pages les plus lues, et celles par lesquelles on arrive. Chaque page est un lien vers ses lignes de log.
- `sources`, `campaigns` — d'où viennent les visites, et les campagnes repérées dans les liens.
- `browsers · systems · devices`, `hour of day` — le profil de l'audience et ses heures, dans votre fuseau.
- `bots`, `status`, `broken links` — les robots identifiés, les codes de réponse, les liens cassés et leur provenance.

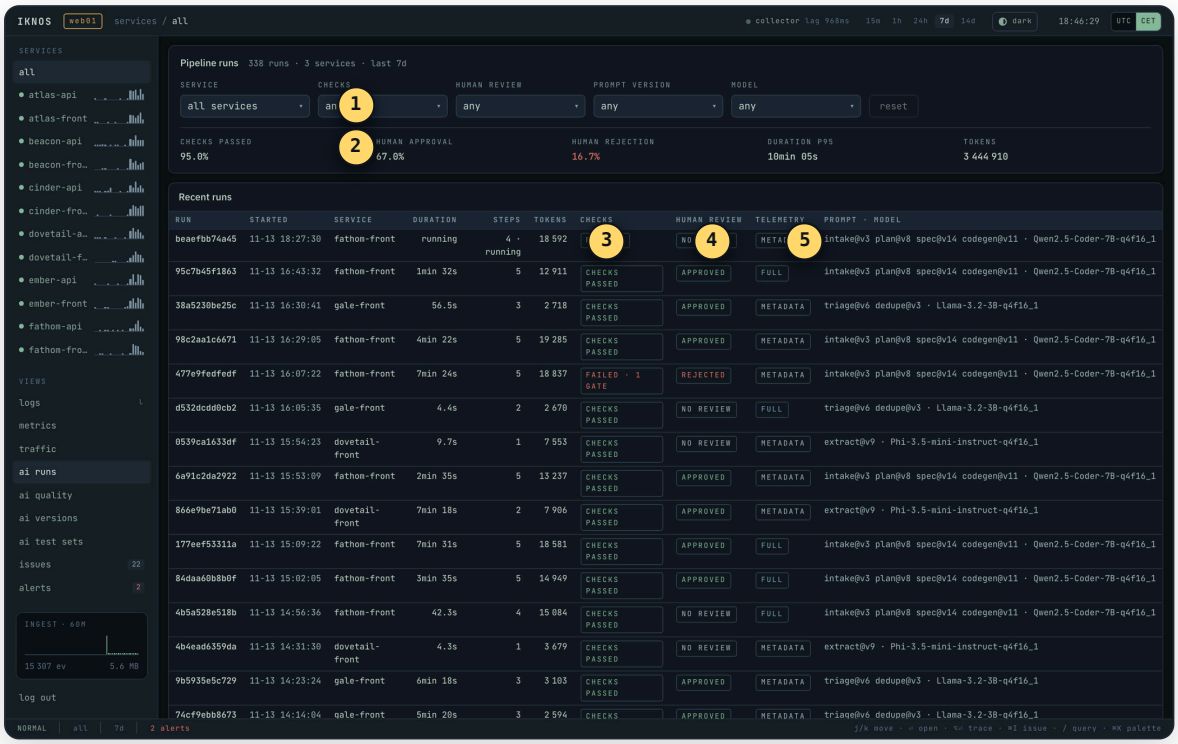
À savoir

- `raw lines in logs`, en haut à droite, ouvre les lignes brutes du site dans la vue logs.
- Un service qui n'est pas un site affiche `This service is not a website nginx serves`.
- Au-delà de la rétention, les requêtes ont disparu : la vue le rappelle (`older requests are gone, not zero`).
- Sur une plage très chargée, la mention `every number is a floor` prévient que les chiffres sont des minimums.

10 Exécutions IA

Certaines applications de la flotte exécutent des pipelines IA et en envoient la trace. Ici, un *run* est une exécution complète ; un *span* est une de ses étapes minutées.

La liste des exécutions AI RUNS

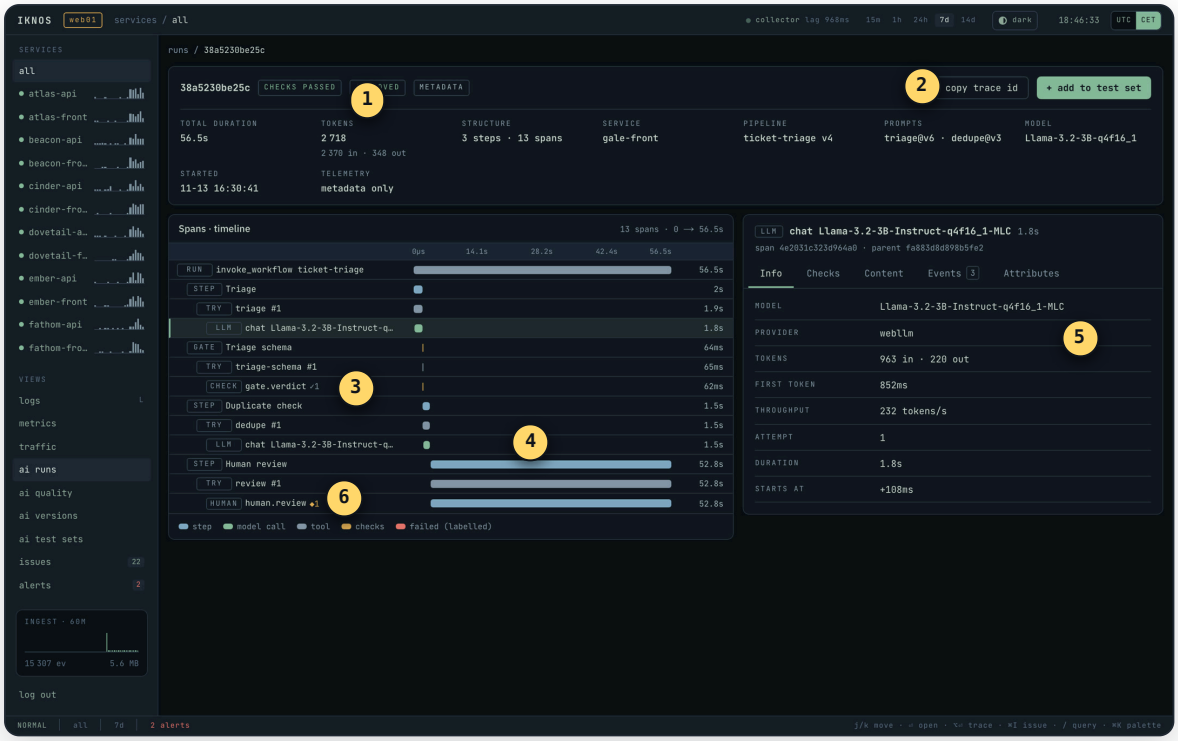


CAPTURE 16 `Pipeline runs`, toutes applications confondues.

1. **Filtres** – `service`, `checks`, `human review`, `prompt version`, `model` ; `reset` les efface.
2. **Indicateurs** – contrôles réussis, approbation et rejet humains, durée p95, jetons consommés.
3. **`checks`** – le résultat des contrôles automatiques : `checks passed`, `failed · 1 gate`, `pending`.
4. **`human review`** – la décision d'un relecteur : `approved`, `edited`, `rejected` ou `no review`.
5. **`telemetry`** – `full` : prompts et réponses capturés. `metadata` : durées, jetons et verdicts seulement.

Autres colonnes – identifiant, début, service, durée (ou `running`), étapes, jetons, version de prompt et modèle. Pagination en pied.

Le détail d'une exécution CLIC SUR UNE LIGNE

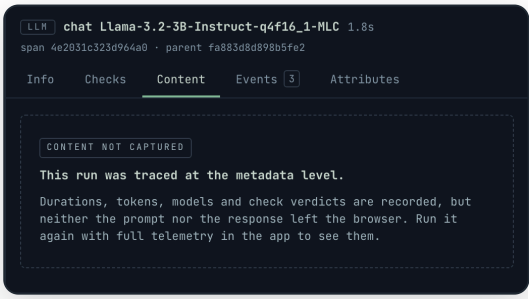


CAPTURE 17 Une exécution du pipeline `ticket-triage` : 3 étapes, 13 spans.

1. **En-tête** – identifiant et badges (contrôles, relecture, télémétrie), puis la synthèse : durée totale, jetons entrée/sortie, structure, service, pipeline, prompts, modèle.
2. **Actions** – `copy trace id`, et `+ add to test set` pour garder cette exécution comme cas de test.
3. **Cascade** `Spans · timeline` – l'arbre de l'exécution, emboîté par parent. Types : `run`, `step`, `gate`, `try`, `llm`, `tool`, `check`, `human`.
4. **Barres** – chaque span sur l'horloge de l'exécution : sa position est son début, sa longueur sa durée. La légende sous la cascade donne les couleurs.
5. **Inspecteur** – le span sélectionné, en cinq onglets (ci-dessous).
6. **Marques** – sur une ligne : ✕ contrôle échoué, ✓ contrôle réussi, ◆ décision humaine. Survoler prévisualise, cliquer épingle le span.

Les cinq onglets de l'inspecteur

ONGLET	CONTENU
`Info`	Modèle, fournisseur, jetons, délai avant le premier jeton, débit en jetons/s, numéro de tentative, durée, instant de départ.
`Checks`	Les verdicts des contrôles (*gates*) concernant ce span : règle, réussite ou échec, score, raisons.
`Content`	Pour un appel de modèle (`llm`) : instructions système, prompt, réponse, avec copie.
`Events`	La chronologie : départ, contrôle échoué, nouvelle tentative, relecture humaine, fin.
`Attributes`	Tous les attributs bruts du span.



CAPTURE 18 L'onglet `Content` d'une exécution tracée en métadonnées.

QUAND LE CONTENU MANQUE

`Content not captured` : l'exécution a été tracée au niveau métadonnées, ni le prompt ni la réponse n'ont quitté le navigateur. `Content expired` : le contenu avait été capturé, puis purgé à la fin de sa durée de conservation – les durées, jetons et verdicts restent.

11 Qualité, versions, jeux de test

Trois vues pour suivre un pipeline IA dans la durée. Les deux premières se lisent pour un service à la fois : choisissez-le dans le rail.

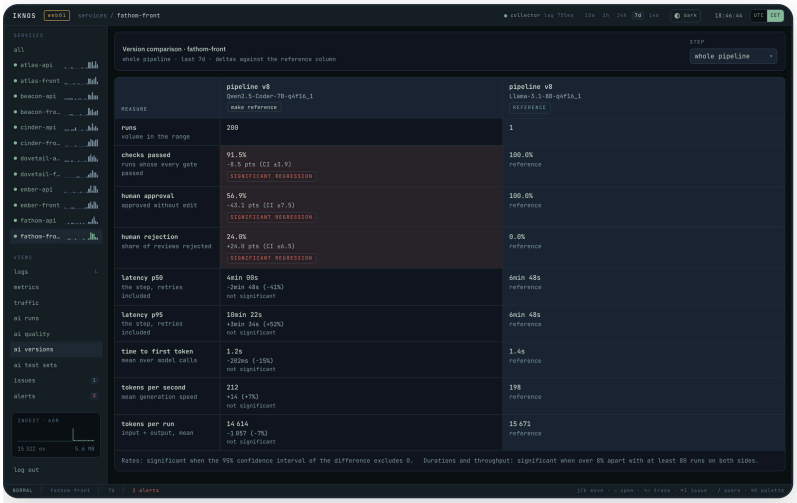
Qualité AI QUALITY



CAPTURE 19 La qualité du pipeline de `fathom-front` sur sept jours.

1. **Quatre indicateurs** – exécutions dont tous les contrôles passent, approbation humaine sans modification, nombre d'exécutions, durée p95. Sous chaque valeur, l'écart avec la fenêtre précédente de même durée.
2. **Marqueurs** – traits verticaux posés aux changements de version de prompt, de modèle ou de pipeline.
3. **`Daily rates`** – par jour, le taux de contrôles réussis (trait plein) et d'approbation humaine (pointillé). Une baisse après un marqueur désigne le changement responsable.
4. **`Failure patterns`** – les contrôles qui échouent, par étape et par règle : nombre d'échecs, part, tendance face à la période d'avant.
5. **`create the ticket`** – crée un ticket Spira pour ce motif d'échec ; une fois créé, la cellule devient un lien vers le ticket et montre son état.

Comparaison de versions AI VERSIONS



CAPTURE 20 Une colonne par version ; les écarts sont calculés contre la colonne de référence.

- Le sélecteur `step` choisit le pipeline entier ou une étape.
- `make reference` désigne la colonne contre laquelle tout est comparé (`REFERENCE`).
- Mesures : exécutions, contrôles réussis, approbation et rejet humains, latence p50 et p95, délai du premier jeton, jetons par seconde, jetons par exécution.
- Une case rouge `significant regression` ou verte `significant improvement` signale un écart qui n'est pas du bruit ; `not significant` dit l'inverse.

La règle est écrite sous le tableau : pour un taux, l'intervalle de confiance à 95 % de la différence doit exclure zéro ; pour une durée ou un débit, il faut plus de 8 % d'écart et au moins 80 exécutions de chaque côté.

Jeux de test AI TEST SETS

Un jeu de test est une collection nommée d'exécutions mises de côté pour servir de cas de non-régression.

- **Créer un jeu** : choisir un service dans le rail, saisir un nom, `create` .
- **Y ajouter une exécution** : depuis son détail, `+ add to test set` , puis choisir un jeu existant ou `a new set…` .
- **Consulter** : cliquer un jeu liste ses exécutions ; `show calls` déplie les appels de modèle avec leur entrée (`input`) et leur sortie de référence.
- **Retirer** : `remove` sur une exécution, `delete set` pour le jeu entier (confirmation demandée, irréversible).

UNE COPIE, PAS UN LIEN

Chaque exécution ajoutée est copiée : son résumé et, si elle était tracée en `full` , les entrées et sorties de ses appels de modèle. La copie survit donc à l'expiration des spans d'origine. Une exécution en métadonnées seules est notée `metadata only` .

PAS DE REJEU DEPUIS IKNOS

Les pipelines tournent dans le navigateur de leurs visiteurs, qu'Iknos ne peut pas atteindre. Les entrées conservées sont les cas à rejouer dans le banc de test de l'application elle-même.

12 Issues

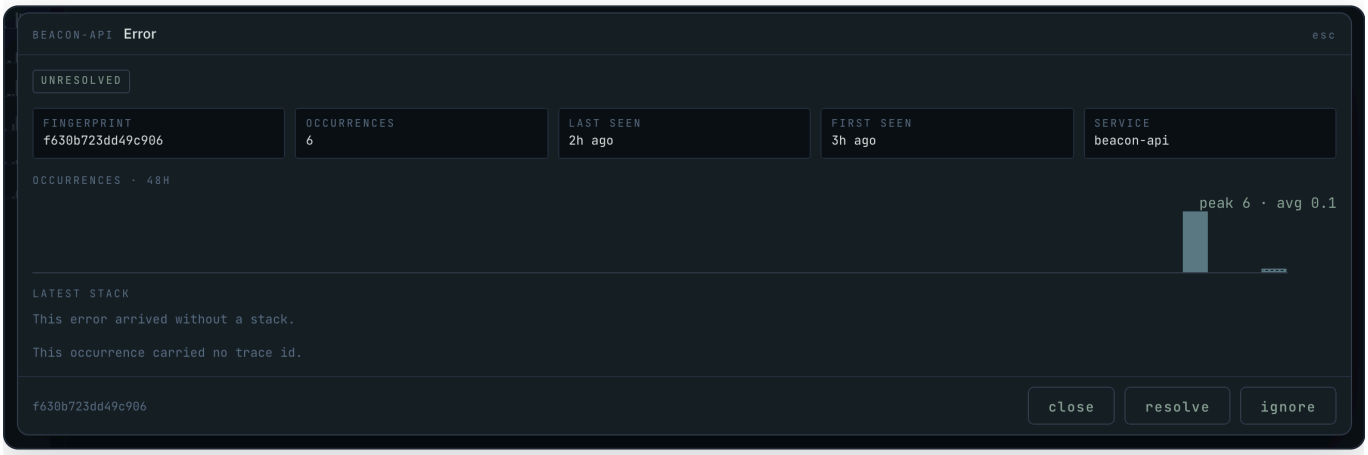
Une issue réunit toutes les occurrences d'une même erreur, reconnue à son empreinte. Cinquante lignes rouges identiques deviennent une ligne et un compteur.



CAPTURE 21 Les issues de `beacon-api` (hauteur recadrée). Avec `all` dans le rail, la liste couvre la flotte.

1. **Segments** – `unresolved`, `resolved`, `ignored`, chacun avec son compte.
2. **Tri** – `last seen` (dernière occurrence), `volume`, `first seen`.
3. **Pastille de fraîcheur et empreinte** – 🔴 vue dans le dernier quart d'heure, 🟠 dans l'heure, 🔵 dans la journée, ⚪ pas aujourd'hui.
4. **Activité** – mini-graphique des occurrences sur 48 h (`events·48h`), total (`count`), première et dernière vue, version (`release`).

La fiche d'une issue CLIC SUR UNE LIGNE · ⌘I DEPUIS UN LOG



CAPTURE 22 Statut, cinq tuiles (empreinte, occurrences, dernière et première vue, service), occurrences sur 48 h, dernière pile d'appels.

- `latest stack` – la pile de la dernière occurrence, quand l'erreur en portait une.
- `open the logs of this request` – ouvre les lignes de la requête fautive, grâce à son trace id.
- `resolve` – l'erreur est corrigée.
- `ignore` – l'erreur est connue et sans intérêt : elle sort de la liste.
- `reopen` – remet une issue résolue ou ignorée parmi les non résolues.

Chaque action est confirmée par un message bref (`Issue resolved.`). Depuis le flux de logs, `⌘` `I` sur une ligne sélectionnée ouvre son issue ; `No issue for this line.` signifie que la ligne n'a pas été groupée.

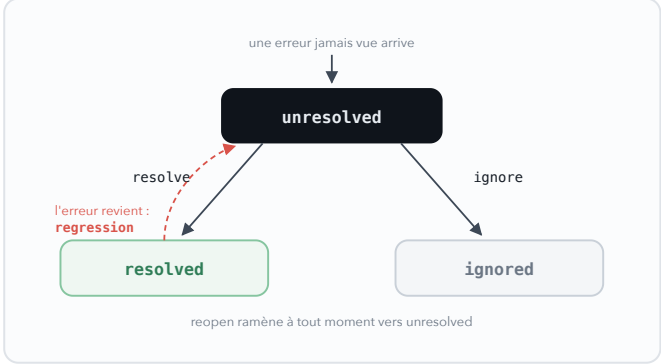
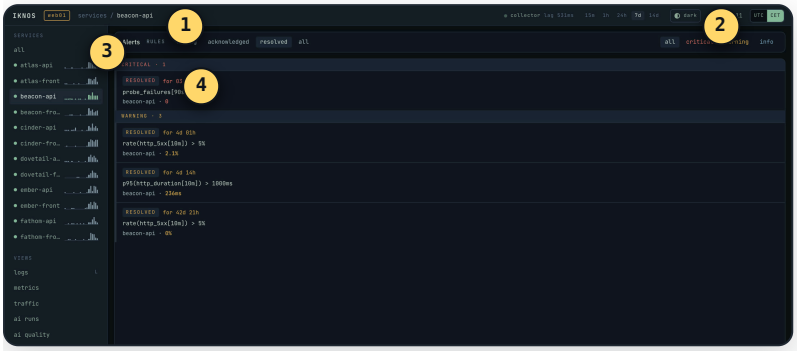


SCHÉMA 3 Une issue résolue dont l'erreur réapparaît redevient non résolue et porte l'étiquette `regression`. Ce n'est pas un quatrième état, mais un drapeau ; la résoudre à nouveau l'efface.

13 Alertes

Des règles évaluées en continu par le collecteur : santé en échec, taux de 5xx, latence, silence d'un service, disque. Rien n'est poussé nulle part — on vient regarder.

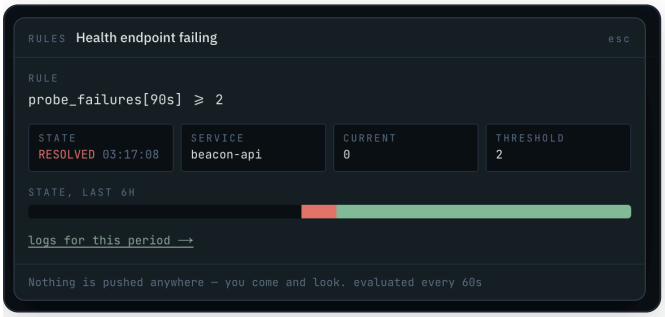


CAPTURE 23 Les alertes résolues de `beacon-api` (hauteur recadrée).

1. **État** — `firing` (en cours, vue par défaut), `acknowledged`, `resolved`, `all`.
2. **Sévérité** — `all`, `critical`, `warning`, `info`.
3. **Groupes** — les cartes sont rangées par sévérité, la plus grave d'abord.
4. **Carte** — badge d'état et durée (`for 4d 01h`), l'expression de la règle, puis le service et la valeur actuelle.

> **UNE LISTE VIDE NE VEUT PAS DIRE « TOUT VA BIEN » PARTOUT**
> Avec un service choisi, la liste ne montre que ses alertes. Si le compteur du rail annonce des alertes et que la liste est vide, elles concernent d'autres services : le message le dit, et `all` dans le rail montre celles de toute la flotte.

La fiche d'une alerte CLIC SUR UNE CARTE



CAPTURE 24 La règle « Health endpoint failing ».

- `RULE` — l'expression évaluée, ici `probe_failures[90s] ⩾ 2`.
- Quatre tuiles : `state`, `service`, `current` (valeur actuelle), `threshold` (seuil).
- `state, last 6h` — une bande colorée retrace les états sur six heures ; survoler un segment donne sa durée et la valeur lue.
- `logs for this period →` — ouvre les logs du service sur la période de l'alerte.
- En pied : la cadence d'évaluation de la règle.

États et actions

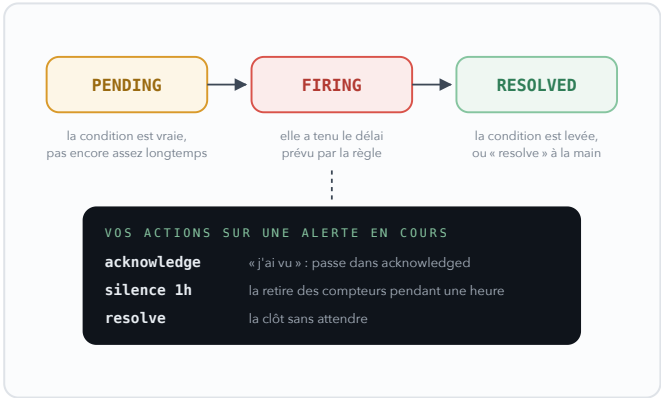


SCHÉMA 4 Acquitter ou faire taire une alerte la range, sans la détacher du réel : elle reste déclenchée et se résout d'elle-même quand la condition disparaît.

Le compteur rouge du rail et celui de la barre d'état comptent les alertes en cours, non acquittées et non réduites au silence. Les alertes encore `PENDING` n'y figurent pas.

14 Palette et clavier

Tout ce qui se fait à la souris se fait au clavier. La barre d'état rappelle les touches en permanence.

La palette ⌘K



CAPTURE 25 `Go to` : on tape, on choisit, on valide.

Elle cherche parmi les services, les issues, les vues, et – dans la fenêtre de temps courante – les routes et les identifiants de trace. Le mot à droite de chaque résultat annonce ce que fera ⏎ :

MOT	EFFET
`scope`	Restreint l'écran à ce service.
`filter`	Pose un filtre (par exemple sur une route).
`open`	Ouvre l'élément (une trace).
`go`	Change de vue.
`issue`	Ouvre la fiche de l'issue.

↑ ↓ déplacent la sélection, ⏎ valide, `esc` ferme.

Tous les raccourcis

TOUCHES	ACTION
`J` / `K` ou ↓ / ↑	Ligne suivante / précédente dans le flux.
⏎	Ouvrir le détail de la ligne sélectionnée.
⌥ ⏎	Ouvrir sa trace.
⌘ `I`	Ouvrir son issue.
⌘ `C`	Copier la ligne en NDJSON (si aucun texte n'est sélectionné ; sinon, copie normale).
`/`	Ouvrir le tiroir de filtre.
⌘ `K`	Ouvrir la palette.
⌘ `L`	Replier / déplier les signaux (logs en pleine hauteur).
⌘ ⇧ `L`	Se déconnecter.
`esc`	Fermer la fenêtre, le tiroir ou la bulle ouverte.
← → `Home` `End`	Parcourir les barres de l'histogramme.

DEUX EXCEPTIONS

Dans un champ de saisie, les lettres vous appartiennent : seul `esc` agit. Dans une fenêtre ouverte (mode `MODAL`), seul `esc` agit aussi – c'est pourquoi le détail d'une ligne offre ses propres boutons `trace` et `issue`.

La barre d'état



CELLULE	SENS
`NORMAL` / `MODAL`	Le mode clavier : tous les raccourcis, ou `esc` seulement parce qu'une fenêtre est ouverte.
`all` · `7d`	Le périmètre et la plage en vigueur.
`tail on` / `tail off`	Le direct est actif ou en pause.
`104 ev / 7d` · `q 52ms`	Le nombre de lignes chargées sur la plage, et la durée de la dernière requête.
`2 alerts`	Les alertes en cours ; un clic ouvre la vue alertes. Absent à zéro.

15 Lire les états vides

Iknos distingue toujours « il n'y a rien », « il n'y a rien à mesurer » et « je n'ai pas pu lire ».
Les mots diffèrent ; voici comment les traduire.

MESSAGE	OÙ	CE QUE CELA VEUT DIRE	QUE FAIRE
`No lines match these filters in this range.`	Logs	Les filtres ne laissent rien passer sur la plage.	Élargir la plage, désactiver un filtre.
`No lines in this range.`	Histogramme	Aucune ligne sur la plage.	Élargir la plage ; vérifier le collecteur.
`No /metrics to read.`	Tuiles de signal	Le service n'expose pas de métriques : c'est un fait permanent, pas une plage à élargir.	Rien – instrumenter l'application.
`No samples in this range.`	Tuiles, routes	Le service est mesuré, mais pas sur cette plage.	Changer de plage.
`One reading — too few for a line.`	Tuiles	Une seule mesure : pas de quoi tracer une courbe.	Attendre, ou élargir.
`No reading in the last few minutes.`	Node runtime	Le processus ne publie plus ses mesures : arrêté, ou en cours de redémarrage.	Regarder `restarts` et la santé.
`No health endpoint is registered for this service.`	En-tête de service	Aucune sonde n'est configurée.	Rien.
`PM2 has reported nothing about this process in the last day`	En-tête de service	Le processus est inconnu de PM2 depuis un jour.	Vérifier qu'il tourne.
`Nothing unresolved for this service.`	Issues	Rien de cassé qui ne soit déjà traité.	–
`No errors have been grouped yet.`	Issues	Le regroupement n'a encore rien produit.	–
`Nothing firing for this service.`	Alertes	Aucune règle déclenchée pour ce service.	Voir `all` pour la flotte.
`The engine has not evaluated anything yet.`	Alertes	Le moteur de règles n'a pas encore tourné.	Patienter.
`Pick a service in the rail`	Métriques, qualité, versions	La vue se lit pour un seul service.	En choisir un dans le rail.
`No run in this range.`	Exécutions IA	Aucune exécution reçue sur la plage.	Élargir la plage.
`No span of this run in the last two weeks`	Détail d'exécution	L'exécution est plus ancienne que la rétention, ou l'identifiant est faux.	–
`No services`	Rail	L'API est injoignable, ou le registre des services est vide.	Vérifier que l'API tourne.
`Could not read …` + `retry`	Partout	La lecture a échoué : ce n'est *pas* une absence de données.	`retry` ; si cela persiste, l'API est en cause.
`reading` suivi de points animés	Partout	Une lecture est en cours.	Patienter.

LA PASTILLE DU COLLECTEUR D'ABORD

Devant un écran vide inattendu, regardez la barre du haut : un collecteur ● en retard ou ● arrêté explique des données absentes bien avant les filtres.

PUIS LA RÉTENTION

Une plage ancienne et vide n'est pas une plage calme. Le panneau `Storage & retention` donne la date de la plus ancienne donnée conservée.

16 Mener une enquête

Les vues sont reliées entre elles : presque chaque chiffre est une porte. Voici le chemin type, d'un signal à la ligne fautive.

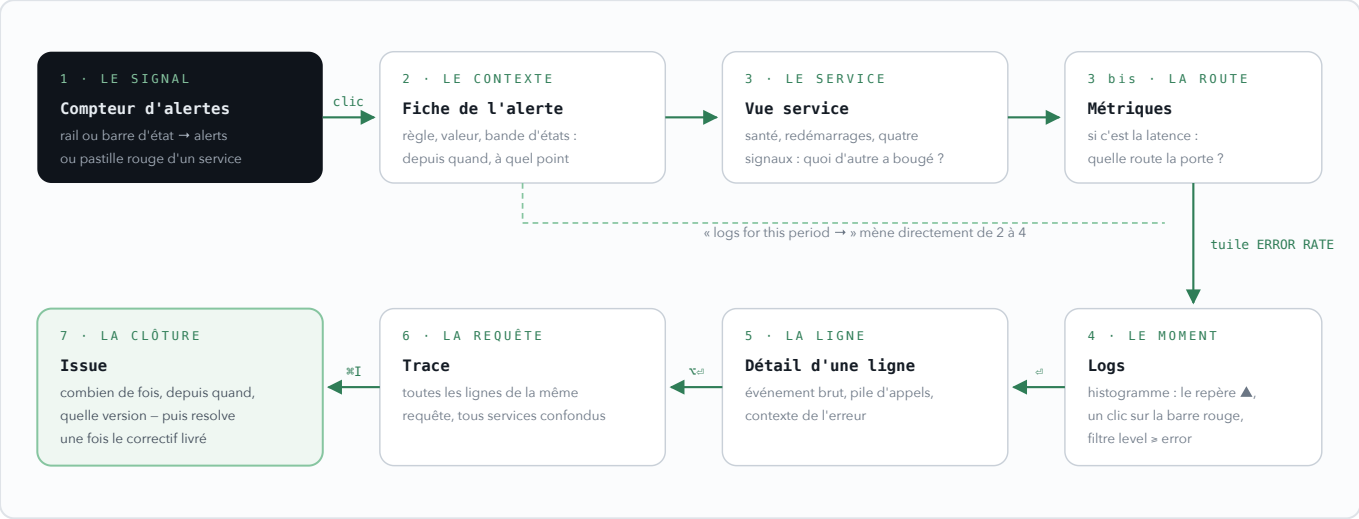


SCHÉMA 5 Le parcours type. Chaque flèche est un clic ou une touche ; on peut entrer par n'importe quelle étape et la palette ⌘ K saute directement à un service, une route, une trace ou une issue.

Toutes les portes, en un tableau

DEPUIS	GESTE	ARRIVÉE
Compteur `alerts` (rail, barre d'état)	clic	Vue alertes.
Fiche d'alerte	`logs for this period →`	Logs du service sur la période de l'alerte.
Tuile `ERROR RATE`	clic	Logs d'erreur du service.
Tuiles `THROUGHPUT`, `p95 LATENCY`	clic	Tableau des routes.
Pastille de santé en échec	clic	Logs d'erreur autour de la sonde ratée.
Route (métriques) · page (trafic)	`logs →` · clic	Lignes de log de cette route.
Barre de l'histogramme	clic	Flux resserré sur l'intervalle.
Ligne de log	clic · ⏎ / colonne trace · ⌥ ⏎ / ⌘ I	Détail / trace / issue.
Fiche d'issue	`open the logs of this request`	Logs de la requête fautive.
Panneaux latéraux de la vue service	clic sur une carte	Fiche de l'alerte ou de l'issue ; le lien en pied ouvre la vue complète.
Motif d'échec (qualité IA)	`create the ticket`	Ticket Spira.
Détail d'une exécution IA	`+ add to test set`	Jeu de test.

EN UNE PHRASE

Un service, une vue, une plage – puis on suit les chiffres : chacun mène à la ligne qui l'explique.